AVL

AVL GROUP AI CHATBOT & SCRAPER SYSTEM

Comprehensive Architecture, Database Design, & Implementation Manual

Document Type:	Developer & Integration Guide
Target System:	Django / PostgreSQL (pgvector) / Docker Compose
Version:	v2.1.0 (PostgreSQL Vector Search Upgrade)
Date of Release:	July 30, 2026
Status:	**Production Ready**

Table of Contents

1. Executive Summary & RAG Architecture

The AVL Group AI Chatbot & Scraper System is a production-ready application designed to parse and index the official Apparels Village Limited (AVL) website (https://avl.com.bd) and provide an intelligent, retrieval-augmented chatbot. The system leverages state-of-the-art vector embedding databases and Large Language Models (LLMs) to deliver contextual responses about AVL's factory, woven/knit fabrics, specifications, certifications, and compliance.

RAG Workflow Overview:

- **Scraping Phase:** A background crawler recursively traverses site URLs. It cleans body elements and parses Elementor image/lightbox gallery titles. Each segment of text is split into semantic paragraphs (max 1200 characters).
- **Embedding Phase:** The text paragraphs are submitted to the OpenAI Embeddings API (model text-embedding-3-small) to generate a 1536-dimensional coordinate vector representing the semantic value of the text.
- **Storage Phase:** Scraped page metrics and chunk vector coords are natively saved to a PostgreSQL database table utilizing the *pgvector* extension.
- **Retrieval & Response Phase:** When a user asks a question, the server embeds the query text and computes a database-level *CosineDistance* query over the index. The top 8 most similar chunks are retrieved and injected as context into the GPT-4o-mini prompt, streaming a response to the UI.

2. Repository Structure Map

The repository is organized following clean Django app layout principles, Docker configuration, and front-end single-page application structure. The key files are listed below:

File / Folder Path	Role / Purpose
/config/	Project root configurations directory.
■■■ settings.py	Global Django settings. Configures conditional DB (PostgreSQL vs SQLite) and API keys.
■■■ urls.py	Root URL router, maps Admin panel, app API paths, and Swagger docs.
/chatbot/	Core application logic folder.
■■■ models.py	Database schemas defining ScrapedPage, PageChunk (pgvector), and ChatSession models.
■■■ views.py	API endpoints for Chat, Scraper triggers, telemetry metrics, and Session controls.
■■■ scraper.py	Recursive BeautifulSoup crawler, embeddings generator, and media description cleaners.
■■■ serializers.py	Django Rest Framework (DRF) serializers defining schema payloads for DRF Spectacular.
■■■ /templates/index.html	Responsive single-page web app. Features ambient background glow and message streams.
Dockerfile	Builds lightweight python:3.11-slim container. Fixes CRLF carriage returns.
docker-compose.yml	Orchestrates Postgres (pgvector) on port 5436, pgAdmin on port 5055, and Web server.
docker-entrypoint.sh	Startup script waiting for DB connection before applying migrations and booting web.
requirements.txt	Declares dependencies, including Django, openai, pgvector, and psycopg2-binary.

3. Scraper Pipeline & Formatter

Standard scrapers often fail on modern pages because important textual details (like certifications) are embedded in interactive image slide galleries and lightboxes rather than plain text headings. Our crawler, implemented in **chatbot/scraper.py**, solves this problem by traversing structural body elements AND parsing lightbox and image metadata attributes.

Media Metadata Cleanup & Formatter:

To make image descriptions semantically searchable, the scraper sanitizes filenames and lightbox title attributes using the **format_extracted_media_info** helper. This expands raw machine strings into clear natural language sentences, heavily improving the similarity embeddings:

```
def format_extracted_media_info(name, url):
    clean = name.replace('_', ' ').replace('-', ' ')
    clean = re.sub(r'page\s*\d+', '', clean, flags=re.IGNORECASE)
    clean = re.sub(r'\bscaled\b', '', clean, flags=re.IGNORECASE)
    clean = re.sub(r'\b\d+\b', '', clean)
    clean = re.sub(r'\s+', ' ', clean).strip(' .()')

    if not clean: return ""
    lower_clean = clean.lower()
    is_cert = 'cert' in url.lower() or any(k in lower_clean for k in
        ['gots', 'oeko', 'grs', 'ocs', 'rcs', 'bsci', 'sedex', 'compliance'])

    if is_cert:
        desc = clean
        if 'gots' in lower_clean and 'global' not in lower_clean:
            desc += " (Global Organic Textile Standard)"
        elif 'oeko' in lower_clean and 'tex' not in lower_clean:
            desc += " (OEKO-TEX Standard 100)"
        return f"Apparels Village Limited (AVL) holds the certification: {desc}."
    return f"Page illustration / Image: {clean}."
```

Sample Cleanups:

Raw Lightbox / Alt Attribute	Post-Sanitization Semantic Output
BSCI_result-2024-page-001	Apparels Village Limited (AVL) holds the certification: BSCI result (Business Social Compliance Initiative).
GOTS_Scope_Certificate_AVL-page-001	Apparels Village Limited (AVL) holds the certification: GOTS Scope Certificate AVL (Global Organic Textile Standard).
OEKO-TEX-Woven-recycled-materials	Apparels Village Limited (AVL) holds the certification: OEKO TEX Woven recycled materials (OEKO-TEX Standard 100).

4. PostgreSQL & pgvector Integration

To achieve production-grade search speeds, the application utilizes PostgreSQL and the **pgvector** extension. Vector calculation is offloaded from python and handled natively in database SQL queries.

Database Model Schema (models.py):

```
from django.db import models
from pgvector.django import VectorField

class PageChunk(models.Model):
    page = models.ForeignKey(ScrapedPage, related_name='chunks', on_delete=models.CASCADE)
    chunk_text = models.TextField()
    embedding = VectorField(dimensions=1536, null=True, blank=True)

    def __str__(self):
        return f"Chunk of {self.page.title}: {self.chunk_text[:30]}..."
```

Database Similarity Search (views.py):

```
from pgvector.django import CosineDistance

def retrieve_relevant_context(query_text, top_k=8):
    try:
        query_vector = get_embedding(query_text)
    except Exception as e:
        return ""

    # Query database natively using pgvector CosineDistance
    top_chunks = PageChunk.objects.order_by(
        CosineDistance('embedding', query_vector)
    )[:top_k]

    if not top_chunks.exists():
        return ""

    context_parts = []
    for chunk in top_chunks:
        context_parts.append(
            f"Source URL: {chunk.page.url}\n"
            f"Page Title: {chunk.page.title}\n"
            f"Content Section:\n{chunk.chunk_text}\n"
        )
    return "\n---\n".join(context_parts)
```

Vector Migrations: To prevent *type 'vector' does not exist* errors, Django migrations execute the VectorExtension() operation to execute CREATE EXTENSION IF NOT EXISTS vector; prior to creating tables. The embedding column is configured with 1536 dimensions corresponding to the OpenAI embeddings array length.

5. API Layer & Stream View

The REST interface is created using Django Rest Framework (DRF) views. Swaggers docs are automatically generated using *drf-spectacular*.

Endpoint Definitions:

Route	Method	Request Payload	Response Description
/api/chat/session/	POST	{'name': 'str', 'email': 'str'}	Initializes session UUID and registers user profile.
/api/chat/	POST	{'message': 'str', 'session_id': 'uuid'}	Retrieves context, invokes OpenAI chat endpoint, and returns answer.
/api/chat/	GET	?session_id=uuid	Restores chat logs and session state.
/api/scrape/	POST	None	Initiates the crawler recursively in a background daemon thread.
/api/status/	GET	None	Returns page count, scraper status (Idle/Active), and crawl timestamp.

Chat Handler Execution Flow (views.py):

1. **ChatView** validates the payload and fetches the corresponding UserChatSession from PostgreSQL.
2. The last 10 messages of the conversation history are loaded from the session's chat_details JSON column.
3. retrieve_relevant_context() executes the pgvector CosineDistance query, returning top 8 chunks.
4. A structured prompt instructions block is created, injecting the database context chunks.
5. The server calls OpenAI's GPT-4o-mini completions endpoint, gets the response, appends the QA pair log, and saves it to the session. The response is returned to the client.

Sample JSON Response:


```
{
  "success": true,
  "message": "Message processed successfully.",
  "data": {
    "session_id": "605f699f98074528afa392261aa10a88",
    "answer": "Apparels Village Limited (AVL) holds several certifications, including Standard 100 by OEKO-TE
X..."
  }
}
```

6. Docker Deployment & Orchestration

The system is containerized to separate the database layer, database UI dashboard, and python django backend allowing standard port exposure and volume persistence.

Docker Compose Service Architecture (docker-compose.yml):

```
services:
  db:
    image: pgvector/pgvector:pg16
    command: postgres -p 5436                   # Run on port 5436 internally
    environment:
      POSTGRES_DB: avl_chatbot
      POSTGRES_USER: chatbot_user
      POSTGRES_PASSWORD: chatbot_password
    ports:
      - "5436:5436"                              # Map internally and externally
    volumes:
      - pgdata:/var/lib/postgresql/data          # Persistent database storage

  pgadmin:
    image: dpage/pgadmin4
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@avl.com
      PGADMIN_DEFAULT_PASSWORD: admin_password
    ports:
      - "5055:80"                                # Expose pgAdmin web interface

  web:
    build: .
    ports:
      - "8005:8005"
    volumes:
      - .:/app                                   # Host directory mount for hot reload
      - /app/venv                                # Keep host virtualenv separate
    environment:
      - DB_HOST=db
      - DB_PORT=5436                             # Connect web to pgvector DB port
    depends_on:
      - db
```

Startup Orchestration (docker-entrypoint.sh):

```
#!/bin/sh
set -e

# Block thread until PostgreSQL port (5436) is open
if [ -n "$DB_HOST" ]; then
  echo "Waiting for PostgreSQL at $DB_HOST:$DB_PORT..."
  python -c "
import socket, time, os
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
while True:
    try:
        s.connect((os.environ.get('DB_HOST', 'db'), int(os.environ.get('DB_PORT', 5436))))
        break
    except socket.error:
        time.sleep(1)
"
fi
python manage.py migrate --noinput
exec "$@"
```

Deployment Commands: Booting the stack is accomplished via docker compose up --build -d. This builds the image, starts the containers, waits for postgres, runs database migrations, and exposes the Django app at http://localhost:8005 and pgAdmin at http://localhost:5055.